Clase 323 — Pruebas de seguridad del software y evaluación

> Parte: **17 — Profundización para certificaciones** · Fuente: *(ISC)² CISSP OSG — Security Assessment & Testing / Software Development Security · OWASP ASVS 4.0* ⏱️ Duración estimada: **140 min** · Nivel: **Avanzado**

🎯 Objetivo

Verificar que el software es seguro **antes** y **después** de desplegarlo. Esta clase cubre las pruebas de seguridad del software desde la óptica de evaluación de CISSP: **revisión de código seguro** (manual y asistida), las técnicas automatizadas **SAST/DAST/IAST/SCA**, las **pruebas de penetración de aplicaciones** (encuadre defensivo y de evaluación), la **gestión de hallazgos** con severidad y SLA, y la **verificación contra OWASP ASVS** como estándar de requisitos. El enfoque es de **evaluación y aseguramiento**: encontrar y gestionar defectos, no explotarlos en producción.

📚 Resultados de aprendizaje

Al finalizar, el alumno podrá:

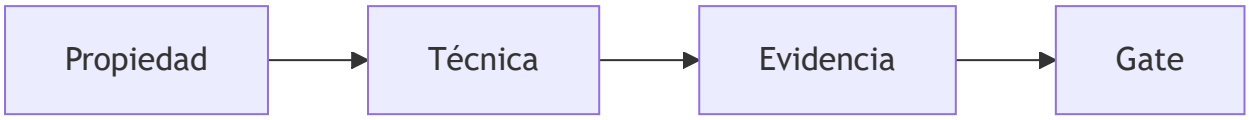
1. **Comparar** SAST, DAST, IAST y SCA, y ubicar cada uno en el SDLC.
2. **Ejecutar** una revisión de código seguro guiada por checklist y por hallazgos de herramienta.
3. **Encuadrar** una prueba de penetración de aplicación: alcance, reglas de enfrentamiento y entregables.
4. **Gestionar** hallazgos: severidad, riesgo, deduplicación, triage de falsos positivos y SLA de remediación.
5. **Verificar** requisitos de seguridad de una aplicación contra los niveles de OWASP ASVS.

🗺️ Temas

#	Tema	Por qué importa
1	Pruebas en el SDLC (shift-left)	Cuanto antes se detecta, más barato se corrige
2	SAST (análisis estático)	Encuentra defectos en el código sin ejecutarlo
3	DAST (análisis dinámico)	Prueba la app en ejecución, como un atacante
4	IAST y SCA	Instrumentación en runtime y dependencias vulnerables
5	Revisión de código seguro	El ojo humano ve lógica que la herramienta no
6	Pentest de aplicaciones	Validación manual de explotabilidad real
7	Gestión de hallazgos y SLA	Convierte reportes en remediación medible
8	OWASP ASVS	Estándar verificable de requisitos de seguridad

🧠 Modelo mental y caso de decisión

La estrategia combina revisión, SAST, DAST, composición y pruebas manuales según propiedad y fase.



Caso razonado. DAST no alcanza un rol; la cobertura inválida el resultado verde y se corrige autenticación.

📓 Glosario operativo complementario

Término	Definición
Evidencia	Información cuya procedencia y relación con un criterio pueden revisarse.
Supuesto	Condición declarada de la que depende una conclusión.
Riesgo residual	Exposición que permanece después del tratamiento.

✅ Criterio de dominio

El alumno demuestra dominio cuando explica el diagrama, resuelve el caso con evidencia, declara límites y puede defender por qué su decisión cambia si cambia un supuesto.

Cómo lo nombra el catálogo CWE y qué cambia al priorizar

El registro consolidado de hallazgos de esta clase tiene una columna «CWE», y rellenarla bien es más trabajo del que parece. Los resultados de SAST traen un CWE sugerido por la herramienta, pero **la etiqueta de la herramienta no es el CWE correcto**: en el laboratorio del programa, Bandit etiqueta `eval` con `CWE-78` cuando el defecto es `CWE-95`, y asigna a un hash débil de contraseña la clase genérica `CWE-327` en lugar de `CWE-916`. El procedimiento correcto es abrir la ficha oficial de la entrada (`python scripts/cwe_kb.py mostrar CWE-95`), comprobar que la descripción encaja con el defecto y registrar el tipo de correspondencia (oficial, inferida o ninguna). Al deduplicar hallazgos de SCA, SAST, DAST y revisión, la combinación causa raíz + ubicación es mejor clave de deduplicación que el texto del aviso.

Al priorizar, la «severidad (CVSS/riesgo)» de la columna del registro no basta: añade KEV, EPSS y exposición con fecha. Además, un SAST puede tener **falsos negativos** demostrables —no detectar un *path traversal* en el laboratorio—, lo que obliga a combinar técnicas, tal como se ve en la clase 370.

📖 Definiciones y características

- **SAST (Static Application Security Testing):** analiza el **código fuente/binario sin ejecutarlo**. Característica clave: cobertura temprana y amplia, pero muchos falsos positivos; no ve fallos de configuración en runtime.
- **DAST (Dynamic Application Security Testing):** prueba la aplicación **en ejecución** desde fuera, sin ver el código. Característica clave: baja tasa de falsos positivos en lo que encuentra, pero cobertura limitada a lo que alcanza; se ejecuta tarde en el ciclo.

- **IAST (Interactive AST):** instrumenta la aplicación y observa su comportamiento **durante las pruebas funcionales**. Característica clave: combina visibilidad de código y runtime, reduciendo falsos positivos.
- **SCA (Software Composition Analysis):** identifica **dependencias y librerías** vulnerables y sus licencias. Característica clave: cubre el riesgo de terceros (la mayor parte del código moderno), p. ej. CVE en paquetes.
- **Revisión de código seguro (secure code review):** inspección manual, guiada por checklist, de zonas sensibles (autenticación, validación de entrada, criptografía, manejo de errores). Característica clave: detecta fallos de **lógica de negocio** que ninguna herramienta automatiza.
- **Prueba de penetración de aplicación:** evaluación autorizada que valida la **explotabilidad real** de los hallazgos. Característica clave: se realiza con alcance y reglas de enfrentamiento por escrito, en entorno autorizado.
- **Positivo/negativo, verdadero/falso:** un **falso positivo** es una alerta sin defecto real; un **falso negativo** es un defecto no detectado. Característica clave: el triage busca minimizar falsos negativos sin ahogarse en falsos positivos.
- **OWASP ASVS (Application Security Verification Standard):** catálogo de requisitos verificables en tres niveles (L1 básico, L2 estándar, L3 crítico). Característica clave: define *qué* verificar y sirve como criterio de aceptación de seguridad.

🧰 Herramientas y preparación

Entorno **de laboratorio propio** con una aplicación deliberadamente vulnerable de práctica:

- **App de práctica** intencionalmente insegura (p. ej. OWASP Juice Shop o WebGoat) desplegada **solo en local/laboratorio**.
- **SAST**: Semgrep o SonarQube sobre el código de la app de práctica.
- **DAST**: OWASP ZAP en modo pasivo/activo contra la app de laboratorio.
- **SCA**: OWASP Dependency-Check o `npm audit` / `pip-audit` según el stack.
- **Checklist ASVS** (OWASP) y una hoja de cálculo de gestión de hallazgos (ID, título, severidad, CWE, estado, SLA).

> Nota ética: todas las pruebas se realizan **exclusivamente** sobre software propio o con autorización explícita por escrito, en entornos aislados. No se prueban aplicaciones de terceros sin permiso: es ilegal.

🧪 Laboratorio guiado — Revisión de código y evaluación de una app de práctica

Ejercicio aplicado: evalúas una aplicación vulnerable de laboratorio combinando revisión manual y herramientas, y gestionas los hallazgos.

1. **Prepara el entorno.** Despliega la app de práctica en una VM/contenedor aislado sin acceso a producción. Confirma el alcance por escrito: qué se prueba y qué queda fuera.
2. **Pasa SCA.** Ejecuta el análisis de dependencias y anota las librerías con CVE conocidos y su severidad. Es el hallazgo más rápido y frecuente.
3. **Ejecuta SAST.** Corre Semgrep/SonarQube sobre el código. Exporta los hallazgos con su CWE y descarta ruido evidente (marcando el criterio del triage).

4. **Haz la revisión de código guiada.** Con checklist ASVS, revisa manualmente cuatro zonas: autenticación/sesión, validación de entrada, criptografía/almacenamiento de secretos y manejo de errores/logging. Documenta hallazgos de **lógica** que SAST no vio.
5. **Ejecuta DAST.** Lanza ZAP contra la app en ejecución (escaneo pasivo y luego activo autorizado). Correlaciona sus hallazgos con los de SAST para confirmar explotabilidad.
6. **Consolida y deduplica.** Une los hallazgos de SCA+SAST+DAST+revisión en un único registro. Marca duplicados y clasifica cada uno con severidad (CVSS/riesgo) y CWE.
7. **Verifica contra ASVS.** Toma un conjunto de requisitos ASVS L1/L2 (p. ej. V2 autenticación, V5 validación, V6 criptografía) y marca cumple/no cumple/parcial con evidencia.
8. **Asigna SLA y dueño.** Define plazo de remediación por severidad (p. ej. crítico 7 días, alto 30) y responsable de cada hallazgo.
9. **Redacta el informe.** Resumen ejecutivo + tabla de hallazgos priorizada + recomendaciones + estado ASVS. Reutiliza lo aprendido en la clase 321 sobre reporte por audiencia.

Entregable: registro consolidado de hallazgos con severidad/CWE/SLA, checklist ASVS verificado con evidencia y el informe de evaluación.

✍️ Ejercicios

1. Sitúa SAST, DAST, IAST y SCA en una línea del SDLC y explica el mejor momento de cada uno.
2. Revisa un fragmento de código dado y encuentra dos fallos de validación de entrada, citando el CWE.
3. Triage de diez hallazgos de SAST: marca falsos positivos con justificación.
4. Mapea cinco hallazgos a requisitos ASVS y determina el nivel (L1/L2/L3) que incumplen.
5. Define una política de SLA de remediación por severidad y justifícala con el riesgo.
6. Correlaciona un hallazgo de SAST con uno de DAST para demostrar explotabilidad real.

📑 Reto verificable

Reto: entrega el informe de evaluación de seguridad de la app de práctica, con hallazgos gestionados y verificación ASVS.

Criterio de aceptación:

- Se combinaron **al menos tres técnicas** (SCA, SAST, DAST y/o revisión manual) sobre software propio/autorizado en entorno aislado.
- El **registro de hallazgos** está deduplicado, con severidad, CWE, dueño y SLA por hallazgo.
- Hay **triage documentado** de falsos positivos con su criterio.
- Se incluye la **verificación ASVS** de un conjunto de requisitos L1/L2 con evidencia de cumple/no cumple.
- El **informe** tiene resumen ejecutivo sin jerga y tabla priorizada de remediación.

⚠️ Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
"SAST reporta 500 hallazgos, imposible revisarlos"	Falta triage y contexto. Filtra por severidad, agrupa por regla y descarta patrones de falso positivo.
"DAST no encontró la vulnerabilidad que sé que existe"	DAST solo ve lo que alcanza. Complementa con SAST y revisión manual (cobertura).
"La herramienta no vio el fallo de lógica de negocio"	Ninguna herramienta razona la lógica. Eso exige revisión manual guiada por checklist.
"Corrimos las pruebas justo antes de producción"	Demasiado tarde. Integra SAST/SCA en el pipeline (shift-left) para corregir barato.
"Reportamos hallazgos pero nadie los arregla"	Sin dueño ni SLA no hay remediación. Asigna responsable y plazo por severidad.
"Probamos una app de un tercero sin permiso"	Es ilegal. Toda prueba requiere autorización por escrito y entorno autorizado.

❓ Preguntas frecuentes

❓ **¿SAST o DAST, cuál elijo?** Ambos: son complementarios. SAST ve el código temprano con muchos falsos positivos; DAST prueba la app viva con menos ruido pero cobertura limitada. Añade SCA para dependencias y revisión manual para la lógica.

❓ **¿Qué nivel de ASVS debo apuntar?** L1 es una línea base para cualquier app; L2 es el objetivo típico de aplicaciones que manejan datos sensibles; L3 se reserva para sistemas de alto valor/vida crítica. Elige según el riesgo del activo.

❓ **¿La revisión manual sigue siendo necesaria con tantas herramientas?** Sí. Las herramientas escalan la búsqueda de patrones conocidos, pero los fallos de lógica de negocio, control de acceso y flujos de autorización casi siempre requieren un revisor humano.

❓ **¿Cómo priorizo qué remediar primero?** Por riesgo real: severidad técnica (CVSS/CWE) combinada con exposición (¿es accesible?), explotabilidad demostrada y valor del activo. No por el orden en que la herramienta los listó.

🔗 Referencias

- Chapple, Stewart & Gibson. *(ISC)² CISSP Official Study Guide*, 9.ª ed., Sybex — *Security Assessment & Testing* y *Software Development Security*.
- OWASP. *Application Security Verification Standard (ASVS) 4.0* — owasp.org/asvs.
- OWASP. *Code Review Guide* y *Web Security Testing Guide (WSTG)* — owasp.org.
- MITRE. *CWE — Common Weakness Enumeration* — cwe.mitre.org.
- OWASP. *ZAP*, *Dependency-Check* y *Juice Shop* — proyectos de referencia.

📥 Material descargable

- 📄 Guía en PDF — versión imprimible de esta clase.
- 🎞️ Presentación (PPTX) — deck para proyectar en clase.

⬅️ Clase anterior

Clase 322 — Threat intelligence operacional avanzada

➡️ Siguiente clase

Clase 324 — Operaciones de seguridad: hardening y gestión de configuración